PROJECT HANDBOOK & INTERVIEW PREP · CONDENSED EDITION

Real-time Fraud Detection MLOps

Kafka → Airflow/MLflow training → Spark Structured Streaming scoring

Repo kanewyp/fraud-detection-mlops · Forward Deployed Engineer interview prep · September 2026

How to read this handbook

NEW Written as if it exists. Build it before you record (Appendix A).

MEASURE A number you must measure yourself. Never quote it before then.

SAY Shaded blocks with a left bar are words to say out loud.

Contents

Part 1 — Understand the project 0

1. The system at a glance 0
2. Architecture 0
3. Data: the transaction simulator 0
4. Training data: snapshot and validation 0
5. Feature engineering: one definition, two runtimes 0
6. The training pipeline (Airflow) 0
7. Experiment tracking (MLflow) 0
8. Model registry and the promotion gate 0
9. Real-time inference (Spark Structured Streaming) 0
10. Training–serving skew: the centerpiece story 0
11. Monitoring and drift 0
12. Observability 0
13. Testing and CI 0
14. Evaluation: metrics, results, and the synthetic-data caveat 0
15. Reproducibility, configuration and security 0
16. Infrastructure and deployment plan (not yet deployed) 0
17. Maturity map 0

Part 2 — Interview prep 0

18. Numbers you can say 0
19. Scripts 0
20. Concepts and design choices 0
21. Operations: quick answers 0
22. Functions worth name-dropping 0
23. Tough follow-ups 0
24. The forward-deployed lens and delivery 0

Appendix 0

A. Build checklist for NEW items 0

B. Pre-recording repo fixes 0

Diagrams 0

Diagram 1 · System architecture 0

Diagram 2 · Training DAG tasks 0

Diagram 3 · Promotion gate: promote_if_better 0

Diagram 4 · One streaming micro-batch: score_batch 0

Diagram 5 · Monitoring DAG 0

Part 1 — Understand the project

1. The system at a glance

An end-to-end MLOps platform for card-fraud detection, running as about **12 containers under Docker Compose**.

Stage	What happens
Simulate	A **producer** simulates credit-card transactions with six injected fraud patterns and streams them to a **Kafka** topic on Confluent Cloud (SASL_SSL).
Train	A daily **Airflow** DAG (CeleryExecutor, 2 workers) snapshots the topic NEW, validates it NEW, and builds **30 features** through a shared module NEW. It tunes a gradient-boosted classifier inside an imbalance-aware pipeline (ordinal encoding → SMOTE → XGBoost, LightGBM or CatBoost, chosen by config) and logs everything to **MLflow** (Postgres metadata, MinIO artifacts).
Promote	A new model takes the `champion` alias only if it beats the current one on the same holdout NEW.
Score	**Spark Structured Streaming** consumes live transactions, computes stateful per-user features from a **Redis** online store NEW, scores with the champion, and publishes alerts (probability + model version) to an output Kafka topic.
Monitor	A daily DAG NEW checks feature and score drift, plus precision and recall against labels that arrive later.

What kind of system this is. Classic *batch training + streaming inference*, not an LLM project, so it contrasts well with Loom and MorningTide. Scoring is **near-real-time, post-authorization alerting** in Spark micro-batches. It is *not* an inline authorization decision; that would need a synchronous, low-latency API (§23).

> **Before anything else:** Kafka credentials are in the public git history. Rotate the key first (Appendix B, item 1).

2. Architecture

Diagram 1 · p. 23 Producer ×2 → Kafka `transactions_v2`. **Training plane:** a bounded snapshot feeds the daily Airflow training DAG, which logs, registers and gates models in MLflow (Postgres metadata, MinIO artifacts). **Serving plane:** Spark Structured Streaming reads the topic, keeps per-user history in Redis, loads `@champion` from MLflow, and writes alerts to Kafka `fraud_predictions_v2` and every score to a MinIO score log. **Monitoring:** a daily DAG reads the score log plus labels (joined by `transaction_id`), logs metrics to MLflow, and can trigger retraining.

Service	Image / build	Port	Role
`postgres`	postgres:13	5432	Airflow metadata + MLflow backend (2nd DB made by `init-multiple-dbs.sh`)
`redis`	redis:7.2-bookworm	internal	Celery broker for Airflow
`airflow-*` (webserver, scheduler, 2 workers, triggerer, init)	`./airflow` (Airflow 2.10.5)	8080	Orchestration
`flower` (profile)	same	5555	Celery monitoring
`minio` + `mc`	minio/minio, minio/mc	9000 / 9001	S3-compatible store; `mc` creates the `mlflow` bucket
`mlflow-server`	`./mlflow`	5500	Tracking server + model registry
`producer` ×2 (1 CPU / 1 GB each)	`./producer`	—	Transaction simulator
`inference`	`./inference` (Spark 3.5.4, Java 17)	8000 (health NEW)	Streaming scorer
`feature-store` NEW	redis:7.2-bookworm	internal	Online per-user history

Design principles

Principle	Where it shows up
Separate training and serving planes	Training is batch (pandas, sklearn/imblearn, Airflow); serving is streaming (Spark). They share only the **feature definition** NEW and the **registered model**.
One feature definition NEW	`common/features.py` is imported by both planes, so no second copy can drift (§10)
Point-in-time correctness	Rolling windows use `closed="left"`, so a transaction sees only strictly earlier ones; the whole-history `user_amount_std` leak is fixed NEW
Imbalance-aware by construction	SMOTE sits *inside* the `imblearn` pipeline, so it resamples training folds only; CV objective is F2; models compared on AUC-PR
Every model is traceable	Params, metrics, plots, signature, dataset URI + hash NEW, feature version NEW, chosen threshold NEW
Promotion is a decision, not a side effect NEW	Registering ≠ deploying. Only the gate moves `champion`; rollback re-points the alias
Config over code	Framework, hyperparameters, topics, risk lists, promotion floors and monitoring thresholds live in `config.yaml`
Everything in containers	`docker compose up -d --build` brings up the whole stack

3. Data: the transaction simulator

`producer/main.py::TransactionProducer` emits a **26-field JSON transaction**: ids (`transaction_id` uuid, `user_id` 1000–9999, `card_id`); amount, currency (USD/EUR/GBP/CAD), merchant and 10 merchant categories with MCC codes; terminal, transaction type, entry mode (CHIP/SWIPE/CONTACTLESS/ONLINE/MANUAL), `timestamp` (UTC now ± 300 s), country and lat/long; device, IP and user agent; card type, brand, expiry and `cvv/zip/address_verified`; and the label **`is_fraud`**.

A transaction is chosen as fraud with probability near the target rate, then one of six patterns overwrites fields:

Pattern (share of fraud)	What it sets
Account takeover (25%)	$500–3000, high-risk merchant, ONLINE, CVV fails, RU/CN/NG. *Fires only for users/cards in the compromised set, so rarely.*
Card testing (20%)	$0.01–4.99, online MCC 5967, ONLINE, CVV and ZIP fail
Merchant collusion (15%)	High-risk merchant, $1000–2500, CHIP/SWIPE, CVV passes
Geographic anomaly (15%)	RU/CN/NG/PK, ONLINE, address fails, $200–1500
Expired card (10%)	Expiry 2022–2024, MANUAL, CVV fails
Verification failure (15%)	All three checks fail, ONLINE, $100–800

- **Rate control:** target 0.5% ± 0.05%. After 100 transactions the base probability is nudged within 0.45–0.55% by the running rate. The committed training logs show **0.73–0.79%**, from runs that predate the current settings.
- **Kafka producer:** gzip, `linger.ms=5`, `batch.size=16384`; SASL_SSL/PLAIN when credentials exist, else PLAINTEXT; keyed by `transaction_id`; delivery-report callback; graceful shutdown flushes for up to 30 s.
- **Validation:** every event is checked against `TRANSACTION_SCHEMA` (JSON Schema + date-time `FormatChecker`); invalid ones are dropped with a warning.

The synthetic-data caveat. Say it before you're asked. The fraud patterns are deterministic rules, and several features encode those same rules, so the model partly learns to recover the simulator. The metrics prove the *platform* works end to end, not real-world detection skill. The honest next step is the same pipeline on a public benchmark such as IEEE-CIS (§14).

4. Training data: snapshot and validation NEW

Before: `_read_data_from_kafka` read from `earliest` until 10 s passed with no new message. While the producer runs, that gap may never happen, so reads were **unbounded** (memory keeps growing) and **non-reproducible** (whatever the topic held at that moment).

Now, `FraudDetectionTraining.snapshot_topic`:

1. Assign all partitions of `transactions_v2` and capture `end_offsets()` **at start**.
2. Seek to the beginning; read until every partition reaches its captured end offset. Bounded and deterministic.
3. Write Parquet to MinIO: `s3://datasets/transactions_v2/snapshot=<UTC ts>/part-0.parquet`.
4. Log the URI, row count, per-partition offset ranges and a **SHA-256** of the file to MLflow. Any model traces back to the exact records it saw.

`common/data_validation.py::validate_training_data(df)` returns a report, logged as `data_validation.json`:

Check	Hard failure (DAG fails)	Warning
Required columns present, dtypes parseable	✔	
`is_fraud` ∈ {0, 1}; positives ≥ 10	✔	
Fraud rate within [0.1%, 5%]	✔	
`amount` within [0.01, 100000]; no null `user_id` / `timestamp`	✔	
Duplicate `transaction_id` rate	> 1%	> 0
Null rate per column		> 1%
Timestamp span ≥ 24 h (so 24 h / 7 d windows mean something)		< 24 h

Why: bad data is the most common silent ML failure. A producer bug that flips every label to 0 would otherwise train a "model" with perfect accuracy.

5. Feature engineering: one definition, two runtimes NEW

Before: features were written twice, in pandas (`_create_features`) for training and in Spark (`add_features`) for inference, and the copies had drifted (§10). **Now** `common/features.py` is the single source of truth: `build_features(df, config)` is a pure pandas function, `FEATURE_COLUMNS` is the ordered list of 30 inputs, and `FEATURE_VERSION = "v3"` is logged with every model and checked at serving time.

Group	Features	Stateful?
Temporal	`transaction_hour`, `is_night` (22:00–05:00), `is_weekend`, `transaction_day` (day of **week** NEW), `weekend_night`	no
Recency / velocity	`time_since_last_transaction` (s), `user_activity_24h`, `high_velocity` (> 3 txns in 1 h)	**yes**
Monetary	`amount`, `rolling_avg_7d`, `amount_to_avg_ratio`, `is_micro_transaction` (< $5), `is_large_transaction` (> $1000), `is_very_small` (< $1)	partly
Anomaly	`amount_outlier` (|z| > 2.5 vs trailing 7-day mean and std NEW)	**yes**
Merchant	`merchant` (ordinal-encoded), `merchant_risk` (config list), `high_risk_category` (MCC 6011/5967/7995)	no
Card / verification	`verification_score` (0–3), `high_risk_entry_mode`, `is_card_expired`, `high_risk_country` (RU/CN/NG/PK/IR), `high_risk_transaction_type`, `non_usd_currency`	no
Pattern flags	`card_testing_pattern`, `takeover_pattern`, `collusion_pattern`, `geo_anomaly_pattern`, `expired_card_pattern`, `verification_fail_pattern`	no

```
def _trailing(df, window, agg):     # per-user rolling aggregate over STRICTLY EARLIER txns
    return (df.set_index("timestamp").groupby("user_id")["amount"]
              .rolling(window, closed="left").agg(agg)).to_numpy()   # df pre-sorted by (user_id, timestamp)

def build_features(df, config):
    df = df.sort_values(["user_id", "timestamp", "transaction_id"]).reset_index(drop=True)
    df["time_since_last_transaction"] = df.groupby("user_id")
["timestamp"].diff().dt.total_seconds().fillna(0.0)
    df["user_activity_24h"] = np.nan_to_num(_trailing(df, "24h", "count"))
    df["txn_count_1h"]      = np.nan_to_num(_trailing(df, "1h", "count"))
    df["rolling_avg_7d"]    = np.nan_to_num(_trailing(df, "7D", "mean"), nan=DEFAULT_AVG_AMOUNT)
    df["user_amount_std"]   = np.nan_to_num(_trailing(df, "7D", "std"))   # was whole-history std: LEAK
```

Two leakage details worth explaining:

1. **`closed="left"`** keeps the current transaction out of its own window. Otherwise `user_activity_24h` would count the transaction being scored, and a large amount would inflate its own average. The original code already did this.
2. **`user_amount_std`** was `groupby("user_id")["amount"].transform("std")`: the std over *all* of the user's transactions, **including future ones**. That is look-ahead leakage, and it can't be computed at serving time. It is now a trailing 7-day std, identical offline and online.

6. The training pipeline (Airflow)

Diagram 2 · p. 24 Task order: `validate_environment` → `snapshot_data` NEW → `validate_data` NEW → `train_and_evaluate` → `gate_and_promote` NEW → `cleanup_resources` (runs on `all_done`).

DAG `fraud_detection_training_v2`

- Schedule `'18 0 * * *'` = **00:18 UTC daily**. The code comment wrongly says 18:00.
- `catchup=False`; tags `fraud`, `ML`; a `doc_md` description.
- Snapshot URI and run id pass between tasks via XCom.
- `cleanup_resources` removes temp pickles and always runs (`trigger_rule='all_done'`).

Hardening NEW

- `retries=1`, `retry_delay=5 min` (logs show manual re-attempts).
- `execution_timeout=2h`.
- `max_active_runs=1` moved into `DAG(...)`; in `default_args` it was silently ignored.

```
preprocessor = ColumnTransformer([("merchant_encoder", OrdinalEncoder(handle_unknown="use_encoded_value",
                                  unknown_value=-1, dtype=np.float32), ["merchant"])],
remainder="passthrough")
pipeline = ImbPipeline([("preprocessor", preprocessor),
                        ("smote", SMOTE(random_state=seed)),     # resamples ONLY when fitting
                        ("classifier", self._create_model())],   # xgboost | lightgbm | catboost
                       memory="./cache/")
searcher = RandomizedSearchCV(pipeline, self._get_param_grid(), n_iter=10,
                              scoring=make_scorer(fbeta_score, beta=2, zero_division=0),  # recall-weighted
                              cv=TimeSeriesSplit(n_splits=3),   # [new] was StratifiedKFold(shuffle, no seed)
                              refit=True, error_score="raise", random_state=seed)
```

Model factory (`_create_model`): XGBoost with `eval_metric="aucpr"`, `tree_method="hist"`, `reg_lambda=1.0`; LightGBM with `objective="binary"`, `metric="auc"`; CatBoost with `eval_metric="AUC"`. Each has its own search space in `_get_param_grid()`. `config.yaml` currently selects **LightGBM**; the three logged runs used **XGBoost**.

Splits and threshold (fixed NEW)

Step	Before	After
Train/test split	Random stratified 80/20	**Time-based:** oldest 64% train, next 16% **validation**, newest 20% **test**. No future leaks into the past; the test set looks like "tomorrow."
CV for tuning	`StratifiedKFold(3, shuffle=True)`, no seed (not reproducible)	`TimeSeriesSplit(3)` inside the training slice
Threshold	Max F1 on **training-set** predictions (in-sample, optimistic)	Max **F2** on **validation**, subject to `precision ≥ promotion.min_precision` (bounds alert volume)
Metric names	`'f1'` was really F-beta, β = 2	Renamed `f2`; added `alert_rate`, `roc_auc`
Serving threshold	Hard-coded `0.70` in inference	Saved as a **model-version tag** and read by inference

Logged artifacts: sample counts, positives, class ratio and test metrics; best hyperparameters and framework; confusion-matrix and PR-curve PNGs; `log_model` with a signature and `registered_model_name=f"fraud_detection_{framework}_model_v2"`; NEW `reference_stats.json` (per-feature bin edges + proportions and the validation score histogram, the drift baseline), dataset URI + hash, `feature_version`, `data_validation.json`.

Cost: logged runs took **~9–55 min** for 58k–100k records with `n_jobs=1`: 10 iterations × 3 folds = 30 fits + a refit, with SMOTE on every fit.

7. Experiment tracking (MLflow)

- **Server:** port 5500; backend store Postgres (`postgresql+psycopg2://…/mlflow`); artifact root `s3://mlflow` on MinIO via `MLFLOW_S3_ENDPOINT_URL`.
- **Bucket bootstrap:** `mc` waits for MinIO and runs `mc mb minio/mlflow`; `_check_minio_connection()` also creates it if missing (belt and braces).
- **Experiment** `fraud_detection_v2`: one MLflow run per DAG run. Each successful run registers a **new version** of `fraud_detection_{xgboost|lightgbm|catboost}_model_v2`.
- **To show in an interview:** two runs side by side in the MLflow UI, comparing AUC-PR, threshold, PR curve and the dataset snapshot each used NEW.

8. Model registry and the promotion gate NEW

Before: every run registered a version, but nothing decided which one served. Inference loaded whichever pickle was last written to the shared `./models` volume, so **the last run to finish went to production**.

Diagram 3 · p. 25 `promote_if_better` runs as its own DAG task. **No champion yet** → set the alias. **Same `feature_version`** → score the champion on the *same* test slice and promote only if challenger AUC-PR ≥ champion − tolerance **and** precision ≥ `min_precision`. **Changed `feature_version`** → promote only if the absolute floors pass (`auc_pr ≥ min_auc_pr`, precision ≥ `min_precision`). Promotion tags `gate=promoted` and `previous_champion=vN`; otherwise the champion stays and the challenger is tagged `gate=rejected` with a reason.

```
promotion:              # config.yaml
  min_auc_pr: 0.50      # absolute floor
  min_precision: 0.30   # bounds analyst alert volume
  tolerance: 0.01       # challenger may be at most this much worse on AUC-PR (noise band)
```

Serving side, `ChampionCache`: at most every `refresh_seconds` (300), inference calls `get_model_version_by_alias(name, "champion")`. If the version changed, it loads `models:/{name}@champion` and reads

the `threshold` and `feature_version` tags. It **refuses** a model whose `feature_version ≠ FEATURE_VERSION`, logs an error and keeps the old model.

Why it works: deploy is a pointer move (no restart, no copied files); rollback is one command (re-point `champion` to `previous_champion`); every served prediction carries `model_version`, so any alert traces back to a run, a dataset snapshot and a threshold.

9. Real-time inference (Spark Structured Streaming)

`inference/main.py::FraudDetectionInference`. Multi-stage image: the builder has Spark 3.5.4 + Hadoop 3 with `spark-sql-kafka-0-10`, `kafka-clients` 3.6.1, token provider and `commons-pool2` jars plus a venv; the runtime has a Java 17 JRE and a non-root `sparkuser`. It runs `spark-submit --conf spark.sql.shuffle.partitions=2` in local mode.

Diagram 4 · p. 26 Per micro-batch: parse the Kafka JSON with a 25-field schema (no `is_fraud`), keeping `kafka_ts` → fetch 7-day history for the batch's users from Redis (pipelined `ZRANGEBYSCORE`) → `build_features(history + batch)`, keeping batch rows → get champion, threshold and version from `ChampionCache` → score, alerting if probability ≥ threshold → alerts to Kafka (keyed by `transaction_id`, no card/IP/device) and all scores to MinIO (path keyed by `batch_id`) → `ZADD` the batch into Redis and trim anything older than 7 days → log batch metrics.

- **Kafka source:** `subscribe=transactions_v2`, `startingOffsets=latest`, SASL_SSL via JAAS. An explicit 25-field `StructType` with **`is_fraud` deliberately absent**, so labels can't leak into serving. The Kafka message timestamp is kept as `kafka_ts` NEW.
- **Scoring, before:** a `@pandas_udf('int')` over 30 columns with a broadcast model and a **hard-coded 0.70 threshold**, writing `prediction == 1` rows as `to_json(struct(*))`.
- **Scoring, now:** `foreachBatch(score_batch)`, so one Python function per batch can read and update Redis, apply the *same* `build_features`, write two sinks and hot-reload the model. `toPandas()` on the driver is fine at demo volume; the scale-out path is `mapInPandas` per partition (§16).

`UserHistoryStore` NEW	Redis command
Key	`user:{user_id}:txns`, a sorted set; score = event epoch; member = `"{transaction_id}:{amount}"`
Read	Pipelined `ZRANGEBYSCORE key (min_batch_ts − 7d) +inf`. Out-of-order events are fine: `build_features` sorts and uses `closed="left"`
Write	`ZADD`. Re-adding a member is a no-op, so **replaying a batch is idempotent**
Retention	`ZREMRANGEBYSCORE key -inf (now − 7d)` + `EXPIRE key 8d`

Outputs. Alerts go to `config.kafka.output_topic` (was hard-coded) with `transaction_id`, `user_id`, `amount`, `merchant`, `timestamp`, `fraud_probability`, `threshold`, `model_version`, `feature_version`, `scored_at`. **No `card_id`, IP, device or user agent** NEW; the old `struct(*)` shipped card numbers downstream. The **score log** NEW writes every score with inputs to Parquet at a path keyed by `batch_id`, so a retried batch overwrites itself.

> **Delivery semantics (know this cold).** Kafka source + checkpoint = **exactly-once reading of input offsets**. `foreachBatch` sinks are **at-least-once** (a failed batch re-runs with the same `batch_id`). So every side effect is idempotent: Redis by member, Parquet by path, Kafka alerts keyed by `transaction_id` for consumer de-duplication. Kafka transactions could give end-to-end exactly-once; alerting doesn't need it.

- **Checkpointing:** `checkpointLocation` from config, now mounted as a volume (`./checkpoints:/app/checkpoints`) NEW; before, it lived in the container and was lost on rebuild. Changing the query shape (UDF → foreachBatch) needs a **new checkpoint directory**.
- **Health** NEW: the Dockerfile `HEALTHCHECK` curled `:8000/health`, but nothing listened, so the container was always "unhealthy." A small `http.server` thread now returns 200 only if the query `isActive` and last progress is < 2 min old.

- **Latency** NEW: the simulator stamps `timestamp` as now ± 300 s, so event time can't measure latency. Latency = `scored_at − kafka_ts`, with p50 and p95 logged per batch MEASURE.

10. Training–serving skew: the centerpiece story NEW

The original inference re-implemented features in Spark and **replaced every stateful feature with a constant** (stateful streaming wasn't implemented; the code comments say so). **7 of 30 inputs** differed at serving time:

Feature	Training (pandas)	Serving before (Spark)	Serving now
`transaction_day`	`dt.day`: day of **month** (1–31)	`dayofweek`: day of **week** (1–7)	Both day of week (0–6) from `build_features`
`time_since_last_transaction`	Real per-user diff	`lit(0.0)`	Redis history → same function
`user_activity_24h`	Rolling 24 h count	`lit(1)`	same
`rolling_avg_7d`	Rolling 7 d mean	`lit(100.0)`	same
`amount_to_avg_ratio`	amount / real avg	amount / 100	same
`high_velocity` (via `txn_count_1h`)	Rolling 1 h count > 3	always 0	same
`amount_outlier` (via `user_amount_std`)	Whole-history std (**leaky**)	`lit(50.0)`	Trailing 7 d std in both

Why it's dangerous: nothing errors. The model sees inputs from a different distribution and quietly scores worse, and velocity and user-deviation features are exactly the ones fraud models rely on most.

Prevention, four layers: (1) **one implementation**, `common/features.py`; (2) a **parity test** (`tests/test_feature_parity.py`): ~2k simulator transactions featurized once offline and once replayed through `score_batch` logic in micro-batches against a fake Redis, asserting `np.allclose` for in-order events; (3) a **version handshake**: `FEATURE_VERSION` tagged on every model, and serving refuses a mismatch; (4) **monitoring**: PSI on stateful features (§11) catches a regression in production.

The one documented difference: a *late* event (older than events already scored for that user) is scored online without history that arrives after it, while offline replay sorted by time sees a slightly different window. Bounded by the ±300 s jitter; acceptable for alerting.

11. Monitoring and drift NEW

Diagram 5 · p. 27 `load_scores` (yesterday's score Parquet) + `load_reference` (the champion's `reference_stats.json`) → `compute_drift` (PSI per feature and on the score); `load_scores` → `join_labels` → `delayed_label_metrics`; both → `log_to_mlflow` → if score PSI > 0.25 or precision drops, fail the task (that is the alert) and trigger retraining.

DAG `fraud_detection_monitoring`, daily at 01:30 UTC.

Drift = Population Stability Index (PSI), per feature and on `fraud_probability`, against the champion's reference bins (validation deciles captured at training time):

$$\mathrm{PSI} = \Sigma_b \, (a_b - e_b) \cdot \ln(a_b / e_b)$$

e = reference share of bin b, a = yesterday's share (+ epsilon for empty bins)

PSI	Meaning
< 0.1	Stable
0.1–0.25	Moderate: warn
> 0.25	Significant: alert, trigger retrain

Performance with delayed labels. In production, fraud labels arrive days or weeks later (chargebacks, investigations). The simulator carries `is_fraud` on the raw event; serving never reads it, but monitoring joins it back **by** `transaction_id`, standing in for a chargeback feed. It computes daily precision, recall, alert rate and AUC-PR, compares them with the champion's test metrics, and alerts beyond `max_precision_drop` / `max_recall_drop`. **Also tracked:** alert volume per hour (analyst load), share of users with no history (cold-start rate), and model-version distribution (confirms a promotion took effect).

12. Observability

Signal	Where
Task status, durations, retries, logs	Airflow UI (:8080); Celery workers in Flower (:5555, profile `flower`)
Runs, params, metrics, plots, versions, aliases, gate tags NEW	MLflow UI (:5500)
Training logs	Python `logging` to stdout and `fraud_detection_model_v2.log`
Producer health	Delivery-report callback; fraud rate logged every 1,000 txns; `get_fraud_statistics()`
Streaming progress NEW	Per-batch JSON line: `batch_id`, rows, alerts, `batch_duration_ms`, latency p50/p95, `model_version`, Redis round-trip; plus Spark `query.lastProgress` rates
Container health	Compose healthchecks (postgres, redis, Airflow components) + inference health endpoint NEW
Drift and production performance NEW	MLflow experiment `fraud_monitoring`, one run per day

Security note: `FraudDetectionInference.__init__` logged `dict(os.environ)` at DEBUG, which would dump every secret if debug logging were on. Removed NEW.

13. Testing and CI NEW

Before: `producer/test_enhanced_producer.py` and `test_fraud_rate.py` were **print-based scripts** with no assertions, a hard-coded `/home/kanewyp/...` path, and a Kafka producer dependency. No CI.

Test file (pytest, `src/tests/`)	What it proves
`test_features.py`	Windows exclude the current txn; perturbing a later amount leaves earlier rows unchanged (no future leakage); column order = `FEATURE_COLUMNS`; stateless flags correct on hand-built rows
`test_feature_parity.py`	Offline `build_features` = replayed micro-batch path through `UserHistoryStore` on `fakeredis`
`test_producer.py`	Each of the six patterns sets its fields (patched `random`); every txn passes the schema; rate controller stays in band over 50k draws; Kafka `Producer` mocked
`test_data_validation.py`	Flipped-label, zero-positive and duplicate-id frames each fail the right check
`test_registry.py`	Promote when better, reject when worse, absolute floors on a feature-version change, alias moves, tags written (MLflow client on a temp SQLite URI)
`test_monitoring.py`	PSI ≈ 0 for identical distributions, > 0.25 for a shifted one; empty-bin epsilon
`test_dag_integrity.py`	`DagBag` loads both DAGs with zero import errors; task ids and dependencies; `max_active_runs == 1`
`test_streaming_smoke.py` (marker **`spark`**)	`score_batch` on a static local-mode DataFrame writes the expected alerts

CI (`ci.yml`, push + PR): `lint` (`ruff check src`) → `unit` (pytest minus `spark`/`airflow` markers; no Kafka, Spark or Airflow) → `dag-integrity` (installs `apache-airflow==2.10.5` with the constraints file) → `docker-build` (all four images, no push). MEASURE Test count and runtime once the suite exists.

14. Evaluation: metrics, results, and the synthetic-data caveat

Metric	Why
AUC-PR (primary)	With ~0.5–0.8% positives, ROC-AUC looks excellent even for weak models because true negatives dominate; PR focuses on the positive class
F2 (tuning objective)	Weights recall 4× precision ($\beta^2 = 4$): a missed fraud usually costs more than a false alert
Precision at threshold	Sets analyst workload; the floor bounds alert volume
Recall at threshold	Share of fraud caught
Alert rate NEW	Capacity planning

Results in the committed Airflow logs (real runs, all XGBoost, all *before* the enhanced features, with the old random split and train-set threshold):

Run (UTC)	Records	Fraud rate	AUC-PR	Precision	Recall	F2 (logged "f1")	Threshold	Time
2025-07-30 03:01	57,990	0.79%	0.471	0.542	0.286	0.316	0.870	~9 min
2025-07-30 04:54	98,208	0.73%	0.566	0.405	0.462	0.449	0.882	~10 min
2025-08-03 12:03	100,062	0.73%	**0.618**	0.454	0.575	**0.546**	0.873	~55 min
Time split + validation threshold NEW			MEASURE	MEASURE	MEASURE	MEASURE		

- **Never quote the README's example metrics** (AUC-PR 0.95, precision 0.89): they are illustrative, not measured. Replace them with the table above (Appendix B).
- **The newer pattern features mirror the simulator's rules** (e.g., `collusion_pattern` needs a high-risk merchant name, which legitimate simulated transactions never use). Post-enhancement metrics will likely jump, and that jump measures *recovering the generator*, not real-world skill.
- **An honest real-data evaluation:** same pipeline on IEEE-CIS or the European cardholder dataset, time-based split, AUC-PR and precision@k MEASURE, and **SMOTE vs `scale_pos_weight` / class weights**; for boosted trees, class weights are often as good and far cheaper MEASURE.

15. Reproducibility, configuration and security

Configuration: `config.yaml` holds non-secrets (MLflow, topics, Spark, framework + params, risk lists, promotion and monitoring thresholds); `.env` holds secrets. NEW Credentials are read **only from the environment**; reading Kafka credentials from `config.yaml` is how they got committed.

Reproducibility: `model.seed` feeds SMOTE, the models and `RandomizedSearchCV`, and CV is now deterministic (`TimeSeriesSplit`). Every model links to an immutable Parquet snapshot with hash and offsets (§4). **Environments were the gap:** every `requirements.txt` was unpinned, and training (Airflow image, Python 3.12) and serving (Python 3.10) resolved packages independently, so a pickled sklearn/imblearn pipeline **isn't guaranteed to load across versions**. NEW Fix: one shared `requirements-ml.txt` pinning scikit-learn, imbalanced-learn, xgboost, lightgbm, catboost, numpy, pandas and mlflow in both images; MLflow also records `requirements.txt` and `python_env.yaml`;

inference loads through the registry with a version check. joblib/pickle execute code on load, so load only from your own registry.

Security area	Current state	Fix
Kafka credentials in git history	Committed in `config.yaml` (and `.env`) earlier; the latest commit uses placeholders, but **history still has them**	**Rotate the Confluent key now**, then purge history (Appendix B)
Tracked files	`.env`, `logs/`, `models/*.pkl`, `__pycache__` were committed before `.gitignore`	`git rm --cached`; keep `.env.example`
Default credentials	Airflow `airflow/airflow` (README says `admin/admin`), MinIO root from `.env`, Postgres `airflow/airflow`, `mlflow/mlflow`	Fine locally; secrets manager in production
Airflow Fernet key	`AIRFLOW__CORE__FERNET_KEY: ''`, so stored connections aren't encrypted	Generate and inject one
Card data in alerts	`to_json(struct(*))` published `card_id` (full Faker card number), IP, device, user agent	Minimized payload NEW; real systems use **tokenized PANs** to stay out of PCI DSS scope
Secrets in logs	`logger.debug(dict(os.environ))`	Removed NEW
Pickle loading	joblib/pickle from a shared volume	Registry-only loading NEW

16. Infrastructure and deployment plan (not yet deployed)

Compose today: one bridge network; Airflow services share the `x-airflow-common` anchor; `airflow-init` migrates the DB and dependents wait on healthchecks; 2 producer replicas (1 CPU / 1 GB) and 2 Celery workers; `docker compose up -d --build` (add `--profile flower` for Flower).

Production target: same architecture, managed or Kubernetes components.

Area	Plan
Images	Push the four images to a registry with immutable git-SHA tags; deploy via Helm or manifests; GitHub Actions builds, tests, pushes
Secrets	Secrets manager, injected as env vars or mounted secrets; never baked into images or config
Streaming	Spark-on-K8s operator or Databricks/EMR; checkpoints in object storage; executors sized to throughput MEASURE; `mapInPandas` per partition with broadcast model and pipelined Redis instead of driver `toPandas()`; partition the topic by `user_id` so a user's events stay ordered
Training	KubernetesPodOperator with right-sized pods and `n_jobs=-1`; bounded window (e.g., last 30 days of snapshots); warm-start once data outgrows memory
Online features	Managed Redis, or Feast (Redis online store + offline store over the Parquet snapshots) for point-in-time joins
Release safety	**Shadow** a new champion (score in parallel, log only, compare alert overlap) → **canary** by user hash → move the alias
Schema governance	Confluent Schema Registry (Avro/Protobuf) instead of JSON + producer-side jsonschema
Alerting	Airflow failure callbacks to Slack/PagerDuty; streaming metrics to Prometheus/Grafana, alerting on lag, batch duration, zero-alert or alert-flood rate

17. Maturity map

Google's MLOps levels: **0** manual training and deployment; **1** automated continuous training with a pipeline and registry; **2** CI/CD for the pipelines themselves. The original repo sat between 0 and 1 (automated training, no deploy decision, no monitoring). With the new items it is a **solid Level 1**, starting Level 2 through CI.

Stage	Status	One-line answer
Streaming ingestion	Strong	Confluent Kafka SASL_SSL, keyed, gzip, schema-validated producer
Snapshot + validation	Built NEW	Bounded offset snapshot to Parquet, hashed, hard + soft checks
Features	Built NEW	30 features, one shared module, point-in-time, leak fixed
Orchestration	Strong	Airflow CeleryExecutor, daily DAG, retries, cleanup `all_done`
Training	Present (+ new)	imblearn pipeline, in-fold SMOTE, 3 frameworks by config, F2 search, time split
Tracking	Strong	MLflow on Postgres + MinIO; metrics, plots, signature, dataset lineage
Registry + promotion	Built NEW	Champion/challenger gate, alias deploy, one-command rollback
Serving	Present (+ new)	Spark Structured Streaming, Redis state, hot reload, PII-minimized alerts
Monitoring	Built NEW	PSI drift, delayed-label precision/recall, retrain trigger
Testing + CI	Built NEW	Features, parity, producer, gate, PSI, DAG integrity, image builds
Security	Needs action	Rotate leaked key, untrack secrets, Fernet key, secrets manager
Deployment	Planned	K8s / managed Spark + Airflow + MLflow, shadow → canary

Part 2 — Interview prep

18. Numbers you can say

Say this	Source	Careful phrasing
30 model features	`FEATURE_COLUMNS`	README says "30+"; the model takes exactly 30
6 injected fraud patterns; target fraud rate 0.5% ± 0.05%	`producer/main.py`	"simulated"
Observed fraud rate 0.73–0.79% in logged runs	Airflow logs	Earlier producer settings
Training sizes 46k–410k transactions per run	Airflow logs	—
Best logged run: AUC-PR 0.62, precision 0.45, recall 0.58 at threshold 0.87 (XGBoost, 100k txns)	Airflow logs, 2025-08-03	Pre-enhancement features, random split; say "on simulated data"
10-iteration random search × 3-fold CV (30 fits + refit)	`train_model`	—
~12 long-running containers; 2 Celery workers; 2 producer replicas	`docker-compose.yml`	—
7 of 30 features skewed at serving; 1 leaked in training	§10	The core MLOps story
Metrics with the corrected protocol	New run	MEASURE
Streaming latency p50/p95, throughput	Per-batch logs	MEASURE from the Kafka timestamp, not event time
Test count	pytest	MEASURE
Never quote: AUC-PR 0.95 / precision 0.89 / "sub-second latency"	README	Illustrative or unmeasured

Resume line consistent with the code, once the new items exist:

> *Real-time Fraud Detection MLOps: built an end-to-end streaming ML platform (Kafka → Airflow/MLflow → Spark Structured Streaming) with 30 point-in-time features shared between training and serving via a Redis online store, imbalance-aware tuning (in-fold SMOTE, F2, AUC-PR), champion/challenger promotion through the MLflow registry, and PSI drift plus delayed-label monitoring. 12-service Docker Compose stack with CI.*

19. Scripts

The 30-second version

> An end-to-end fraud-detection MLOps platform: Kafka streaming, daily Airflow training with in-fold SMOTE and F2 tuning, MLflow tracking with a champion/challenger gate, Spark Structured Streaming scoring with a Redis online feature store, and drift monitoring. The main lesson was training–serving skew: I fixed seven mismatched features with a shared feature module and a parity test.

"Tell me about the project" (about 2:45)

This is an end-to-end MLOps platform for credit-card fraud detection. I wanted to build the whole lifecycle, not just a model: data streaming in, automated training, a deploy decision, real-time scoring and monitoring, all running as about a dozen containers with Docker Compose.

Data comes from a simulator I wrote. It generates realistic transactions, injects six fraud patterns such as card testing, account takeover and merchant collusion at around half a percent, and streams them to Kafka on Confluent Cloud.

Training is a daily Airflow DAG. It takes a bounded snapshot of the topic, captured by offsets and saved as Parquet, so every model is reproducible from exactly the data it saw. It validates that data, then builds thirty features: temporal ones, card-verification signals, and per-user behavioral ones like 24-hour activity and a 7-day average amount, all computed point-in-time so a transaction only sees earlier ones. The model is gradient boosting, XGBoost, LightGBM or CatBoost by config, inside an imbalanced-learn pipeline so SMOTE only resamples training folds. I tune for F2, because missing fraud costs more than a false alert, pick the threshold on a time-based validation slice, and compare models on AUC-PR, since with under one percent positives ROC-AUC is misleading. Everything goes to MLflow, and a new model only takes the champion alias if it beats the current one on the same holdout.

Serving is Spark Structured Streaming. For each micro-batch it pulls each user's recent history from Redis, computes features with the exact same function used in training, scores with the champion, and publishes alerts with the probability and model version back to Kafka. It also logs every score for monitoring. A daily job computes PSI drift, and precision and recall against labels that arrive later, and can trigger retraining.

The most important lesson was training–serving skew. My first version reimplemented the features in Spark and replaced the stateful ones with constants, so seven of the thirty inputs were different in production. Nothing errored; the model would just quietly get worse. I fixed it with one shared feature module, an online state store, a parity test, and a feature-version handshake between the registry and serving.

One caveat I'm upfront about: the data is simulated, and some features mirror the simulator's rules. So the metrics validate the platform, not real-world fraud skill; the next step is a public benchmark. For a customer, what transfers is the platform: lineage, gating, parity and monitoring.

"Walk me through the pipeline" (about 1:50)

Five stages. **Ingest:** two producer replicas stream JSON transactions, keyed by transaction id and schema-validated, to a Kafka topic over SASL_SSL.

Train: Airflow runs daily. Validate the environment; snapshot the topic up to the end offsets captured at start into Parquet in MinIO; run data checks like fraud-rate bounds, duplicates and nulls; then train. Training builds the thirty features with the shared module and splits by time: oldest data for training, then validation, then the newest twenty percent as test. A randomized search over an imbalanced-learn pipeline (ordinal encoding, SMOTE, the booster) scores on F2 with time-series CV. The threshold is chosen on validation with a precision floor. Everything is logged to MLflow and the model is registered.

Gate: compare the challenger with the champion on the same test slice. Promote only if AUC-PR holds within a tolerance and precision clears the floor; otherwise tag it rejected with a reason. Deploying is just moving the champion alias.

Serve: Spark reads the topic. Each micro-batch fetches seven days of per-user history from Redis, runs the same feature function, scores with the cached champion, reloaded when the alias moves, writes alerts to Kafka and all scores to Parquet, then updates Redis idempotently by transaction id.

Monitor: a daily DAG computes PSI for the features and scores against the champion's reference stats, plus precision and recall against late labels, logs it to MLflow, and fails loudly or triggers retraining when thresholds are breached.

20. Concepts and design choices

Shape every answer as **mechanism → where in the code → why → honest limit.**

Q What is training–serving skew, and how did you prevent it?

> Production features differing from training features: different code, data or timing. Nothing crashes; predictions just degrade. My Spark inference re-implemented features and used constants for the stateful ones. The fix is structural: one feature module, `common/features.py`, used by both planes; Redis online state so stateful features can actually be computed; a parity test replaying two thousand transactions through the micro-batch path against a fake Redis; and a `feature_version` tag that serving checks, plus drift monitoring on the stateful features.

Q What is data leakage, and where could it happen here?

> Training information that isn't available at prediction time. Four risks, each fixed: SMOTE before splitting (now in-fold inside `ImbPipeline`); windows that include the current transaction (`closed="left"`); a per-user std over the whole history, including the future (now a trailing 7-day window); and random splits on time-series data (now a time split with `TimeSeriesSplit`). A unit test perturbs a later transaction and asserts earlier rows don't change.

Q Why AUC-PR instead of ROC-AUC?

> At half a percent fraud, true negatives dominate ROC: the false-positive rate stays tiny even with many false alerts, so weak models score around 0.95. PR focuses on the positive class, and its baseline is the fraud rate itself, about 0.005, so 0.6 is a big lift. Precision also maps directly to analyst workload.

Q How did you handle imbalance? SMOTE or class weights?

> Four layers: in-fold SMOTE, an F2 search objective, AUC-PR as the headline metric, and a validation-chosen threshold with a precision floor. For boosted trees, `scale_pos_weight` is often as good and far cheaper (SMOTE dominates my 30-fit search time), and SMOTE distorts calibration. I'd benchmark both on real data and keep the cheaper one if they tie.

Q How do you choose the threshold, and where does it live?

> Not 0.5: after imbalance and SMOTE, probabilities aren't calibrated. I maximize F2 on the validation slice, subject to a precision floor that caps alert volume; in production I'd minimize expected cost (missed fraud plus review cost) within analyst capacity. It's stored as a model-version tag that serving reads. Before, inference hard-coded 0.70 while training picked about 0.87, which was another form of skew.

Q What is stateful stream processing here, and how do you compute it?

> Features like "transactions in the last hour" need history that persists across micro-batches. Spark's state store (`applyInPandasWithState`) was an option; I chose Redis because history can be inspected, backfilled and shared, and the feature function stays identical to training. Each batch does one pipelined fetch of seven days per user, runs `build_features` on history plus batch, keeps the batch rows, then `ZADD`s the new transactions and trims anything older than seven days.

Q Exactly-once or at-least-once: what do you guarantee?

> Kafka plus the checkpoint reads offsets exactly once, but `foreachBatch` sinks are at-least-once, because a failed batch re-runs with the same id. So every side effect is idempotent: Redis keyed by transaction id, Parquet

> by a batch-id path, alerts keyed by transaction id for consumer deduplication. That's effectively-once without the overhead of Kafka transactions.

Q Concept drift vs data drift: how do you detect each?

> Data drift is shifting inputs: PSI per feature against the training reference (under 0.1 stable, 0.1–0.25 warn, over 0.25 alert). Concept drift is a changed relationship (fraudsters adapt), so inputs can look stable while performance drops. I detect it by joining delayed labels back by transaction id and tracking daily precision and recall, with score-distribution PSI as the early signal, since labels lag by days.

Q What does the registry give you, and how do you decide what goes to production?

> Lineage from each version to its run, params, metrics and data; aliases that separate "a model exists" from "a model is serving"; auditable decisions. `promote_if_better`: no champion, promote; same feature version, score the champion on the same test slice and promote only if AUC-PR is within tolerance and precision clears the floor; changed feature version, apply absolute floors. Rejections are tagged with a reason. Before, the last job to finish overwrote the pickle and went live.

Q How do you roll back, or update the model without restarting the stream?

> `ChampionCache` checks the alias at most every five minutes, loads the new version, validates its feature version and swaps, with no restart, redeploy or checkpoint change; a failed load keeps the old model. Rollback is re-pointing `champion` to the `previous_champion` tag with one client call, and every alert carries `model_version`, so I can find exactly what a bad model produced.

Q Why Airflow? Why Spark?

> Airflow for scheduled, dependency-ordered batch work with retries, backfills and a UI. Spark for Kafka integration, checkpointed exactly-once reads and scale-out with the same code, which is more than this volume needs. An inline authorization decision would need neither: that's a synchronous, low-latency model API.

21. Operations: quick answers

"How did you..."	One-breath answer
...handle late or out-of-order events?	Features are event-time based and sort before windowing, so a late event uses truly earlier history if it's in Redis. Limit: an event scored before its history arrives sees a slightly different window, bounded by the five-minute jitter; partitioning Kafka by user id keeps each user ordered.
...validate data quality?	`validate_training_data` runs as its own task. Hard failures (missing columns, bad labels, < 10 positives, fraud rate outside 0.1–5%, bad amounts, null ids, > 1% duplicates) stop the run; warnings are logged; the report goes to MLflow. The producer also schema-validates every event.
...make training reproducible?	Offset-bounded Parquet snapshot with its hash; params in MLflow; seeds plus a deterministic `TimeSeriesSplit`; pinned, identical ML libraries in both images; the `feature_version` tag. Before, no run could be reconstructed.
...test ML code?	Pure functions (features, validation, gate, PSI) get exact assertions, including the leakage and parity tests; producer tests patch `random`; a `DagBag` test catches import errors, the most common Airflow outage; CI runs lint, unit, DAG integrity and all four image builds.

"How did you..."	One-breath answer
...orchestrate training?	CeleryExecutor (Redis broker, Postgres metadata, 2 workers), daily at 00:18 UTC, no catchup; cleanup uses `all_done`; one retry after 5 min, a 2 h timeout, and `max_active_runs` moved out of `default_args`, where it was ignored.
...choose among three frameworks?	XGBoost (robust, mature), LightGBM (fastest on large data), CatBoost (native categoricals) behind `_create_model` / `_get_param_grid`; everything else is identical. Logged runs are XGBoost, config selects LightGBM; a fair comparison on one snapshot is cheap MEASURE.
...scale it?	Serving: `mapInPandas` per partition with a broadcast model, the topic partitioned by user id, a Spark cluster, managed Redis. Training: KubernetesPodOperator pods, a parallel search, a bounded window. The architecture doesn't change; the runtimes do.
...secure credentials?	Rotated the leaked key, purged history, env-only secrets with `.env.example`, set the Fernet key, planned a secrets manager; removed the env-dumping debug log and card numbers, IPs and device ids from alerts. *Say it only once it's done.*
...measure latency?	From the Kafka message timestamp to the end of scoring, p50/p95 per batch MEASURE; not the transaction timestamp, which carries up to five minutes of jitter. Spark's `lastProgress` gives throughput MEASURE.

22. Functions worth name-dropping

Function / class	File → one sentence
`TransactionProducer.generate_transaction`	`producer/main.py`: Faker transactions, six patterns, adaptive rate control, jsonschema validation
`FraudDetectionTraining.snapshot_topic` NEW	`dags/fraud_detection_training.py`: bounded read to start-time end offsets → Parquet + hash
`validate_training_data` NEW	`common/data_validation.py`: hard and soft checks, logged to MLflow
`build_features` / `FEATURE_COLUMNS` / `FEATURE_VERSION` NEW	`common/features.py`: the single, point-in-time feature definition (`closed="left"`)
`_create_model` / `_get_param_grid`	Training module: framework by config, each with its own search space
`ImbPipeline([... SMOTE ...])` + `RandomizedSearchCV(make_scorer(fbeta_score, beta=2))`	Training module: in-fold resampling, recall-weighted tuning
`promote_if_better` / `ChampionCache` NEW	`common/registry.py`: champion/challenger gate; alias loading, hot reload, version check
`score_batch` NEW	`inference/main.py`: foreachBatch history → features → score → sinks → state
`UserHistoryStore` NEW	`inference/feature_store.py`: Redis sorted sets, idempotent `ZADD`, 7-day trim
`read_data_from_kafka`	`inference/main.py`: readStream with explicit schema, no label field
`compute_psi` NEW	`dags/fraud_detection_monitoring_dag.py`: drift per feature and on the score

23. Tough follow-ups

Q "Your data is synthetic. Why should I believe these metrics?"

> You shouldn't treat them as real-world performance; they validate the platform. The fraud patterns are rules and some features mirror them, so the model is partly recovering the generator. What I'd show instead is the lineage, the gate, the parity test and the monitoring. Next step: this exact pipeline on a public benchmark like IEEE-CIS with the time-based split.

Q "Aren't the pattern features just cheating?"

> On this data, partly. But in real fraud teams, rule-derived features are legitimate and common, because analysts' rules encode real domain knowledge. The risk is the same: they overfit known patterns and miss new ones. That's why you pair them with behavioral features like velocity and deviation from the user's norm, and why concept-drift monitoring matters.

Q "Why Spark for this volume?"

> At demo volume, it's more than needed; a single Python consumer could score it. I chose it for checkpointed exactly-once reading, Kafka integration and a scale-out path with the same code. For a low-volume customer I'd recommend the simpler service.

Q "Is this real-time enough to block a fraudulent transaction?"

> No. It's near-real-time alerting after authorization, in micro-batches. Blocking sits inline in authorization with a budget of tens of milliseconds: a synchronous model API with precomputed online features. The registry, features and Redis state carry over; the serving runtime changes.

Q "Your README claims sub-second latency and enterprise monitoring."

> Those were aspirational when I wrote them. I've since measured latency from the Kafka timestamp MEASURE, added the monitoring DAG and corrected the README. I'd rather present measured numbers. *Fix the README before you record.*

Q "I found Kafka credentials in your commit history."

> Yes, I found it too. I rotated the key, purged the history, and moved every secret to the environment. The lesson: keep secrets out of config files entirely, and add secret scanning to CI. *Say this only once it's true.*

Q "Why not Kubeflow, or Feast, from the start?"

> Scope. Airflow plus MLflow is the most common open-source stack and runs on a laptop, and Redis gave me the online-state piece of a feature store with little machinery. At scale, Feast on the same Parquet snapshots and Redis adds point-in-time joins and a feature registry: a natural next step, not a rewrite.

Q "What's the biggest weakness?"

> Real-world validity, because of synthetic data. After that: serving scores on the driver, fine at this volume but not at scale; and the gate compares one holdout, where I'd add bootstrap confidence intervals so promotion doesn't flip on noise.

24. The forward-deployed lens and delivery

> The model is the smallest part. What makes fraud ML work at a customer is lineage from every alert back to data and model, a gated deploy, feature parity between batch and real time, and monitoring that works with labels that arrive weeks late.

Discovery questions for a bank or fintech:

1. **Decision point:** inline authorization (a latency budget in ms; fail-open or fail-closed?) or post-authorization case creation?
2. **Labels:** where do they come from (chargebacks, analyst dispositions, disputes), how delayed and how noisy?
3. **Economics:** cost of missed fraud vs a false decline or review, and analyst capacity per day; together they set the threshold.
4. **Data:** which features exist online at decision time? Is PAN tokenization in place (PCI DSS scope)? Data-residency rules?
5. **Governance:** model risk management (e.g., SR 11-7 in US banking), explainability and reason codes for analysts, an audit trail for every deploy.
6. **Integration:** does the model replace the rules engine, run alongside it, or feed it?

Why it transfers: gate tags in the registry are the audit trail; the shared feature module and parity test answer "will production match validation?"; delayed-label monitoring proves value month over month.

Delivery tips (one-way video): lead with the lifecycle, not the model. Tell the skew story as the centerpiece: the problem, why it was silent, the structural fix and the test that proves it. State the synthetic-data caveat yourself, early. Give numbers with provenance ("AUC-PR 0.62 on simulated data, before the feature and protocol fixes"). The 2-minute script runs ~2:45 and the walkthrough ~1:50; trim the simulator paragraph first. End forward-looking: the benchmark evaluation, the scale-out path, the customer lesson.

Appendix

A. Build checklist for NEW items

Order matters. Do A0 (security) today, then A1–A3 (P0: the claims interviews probe hardest), then A4–A6 (P1).

#	Task	Files
A0	**Security and hygiene: do first**	
0.1	**Rotate the Confluent Cloud API key** and purge it from history (Appendix B)	—
0.2	Read Kafka credentials only from env vars; delete `username` / `password` from `config.yaml`	config, training, inference
0.3	`git rm --cached` `src/.env`, `src/logs/`, `src/models/*.pkl`, `src/dags/__pycache__/`; add `.env.example`	repo
0.4	Remove `logger.debug(dict(os.environ))`; set `AIRFLOW__CORE__FERNET_KEY` from env	`inference/main.py`, compose
0.5	Secret scanning in CI (e.g., gitleaks)	`ci.yml`
A1	**Shared point-in-time features + online state (P0)**	
1.1	Move `_create_features` into `build_features`; `_trailing()` with `closed="left"`; day-of-week `transaction_day`; trailing 7-day `user_amount_std`; export `FEATURE_COLUMNS`, `FEATURE_VERSION="v3"`	`common/features.py`
1.2	Training calls `build_features`; log `feature_version` as param and tag	training module
1.3	`UserHistoryStore`: pipelined `fetch_history`, `append` via `ZADD` (member `transaction_id:amount`), 7-day trim, `EXPIRE`	`inference/feature_store.py`
1.4	`feature-store` Redis service + volume; mount or package `./common` in airflow and inference	compose, Dockerfiles
1.5	`pandas_udf` → `foreachBatch(score_batch)`; keep `kafka_ts`; minimized alerts; config output topic; score log to MinIO; checkpoint on a volume	`inference/main.py`
1.6	Tests: `test_features.py` (leakage, perturbation), `test_feature_parity.py` (`fakeredis`)	`src/tests/`
A2	**Correct evaluation + registry gate (P0)**	
2.1	Time split 64/16/20, `TimeSeriesSplit(3)`, F2-on-validation threshold with precision floor; `f1` → `f2`; add `alert_rate`, `roc_auc`	training module
2.2	`infer_signature(X_test, test_proba)`; log `reference_stats.json` (decile bins per feature + score)	training module
2.3	`promote_if_better` + `promotion:` config; alias + tags (`threshold`, `feature_version`, `gate`, `previous_champion`, `dataset_hash`)	`common/registry.py`, config
2.4	`ChampionCache` (300 s refresh, version check, keep old model on failure); inference loads from the registry	`registry.py`, `inference/main.py`
2.5	DAG: snapshot → validate → train → gate → cleanup with XCom; `retries`, `retry_delay`, `execution_timeout`; `max_active_runs` into `DAG(...)`; fix cron comment	training DAG
2.6	`test_registry.py` (SQLite tracking URI)	`src/tests/`
A3	**Snapshot + validation (P0)**	
3.1	`snapshot_topic()`: partitions, `end_offsets()`, read to them, Parquet to `s3://datasets/...`, log URI, rows, offsets, SHA-256	training module
3.2	`validate_training_data(df) -> report`, logged to MLflow; `test_data_validation.py`	`common/data_validation.py`

#	Task	Files
A4	**Monitoring (P1)**	
4.1	`compute_psi(ref_bins, actual, eps)`; tasks load_scores → … → check_thresholds (+ `TriggerDagRunOperator`)	monitoring DAG
4.2	`monitoring:` config (`psi_warn=0.1`, `psi_alert=0.25`, `max_precision_drop`, `max_recall_drop`); `test_monitoring.py`	config, tests
A5	**Observability + health (P1)**	
5.1	Per-batch JSON metrics (rows, alerts, duration, p50/p95 vs `kafka_ts`, version, Redis RTT); health thread on :8000	`inference/main.py`
5.2	MEASURE latency and throughput at a known producer rate (`interval` > 0)	—
A6	**Tests, CI, reproducibility (P1)**	
6.1	`test_producer.py` replaces print scripts; `test_dag_integrity.py`	`src/tests/`
6.2	`ci.yml`: lint, unit, dag-integrity (constraints file), docker-build, gitleaks	`.github/workflows/`
6.3	Shared pinned `requirements-ml.txt` for airflow and inference; `imblearn` → `imbalanced-learn`	requirements
6.4	Re-run training with the corrected protocol; fill in the §14 and §18 values	—

B. Pre-recording repo fixes

1. Credentials in public git history. Earlier commits contain the Confluent Cloud API key and secret in `src/config.yaml`, and values in `src/.env`; the latest commit only replaced them with placeholders. This handbook deliberately doesn't reproduce them. In order:

1. **Revoke and rotate** the key in Confluent Cloud now, and review the cluster's audit and usage logs. Rotation is what actually protects you; clones and forks may already hold the old history.
2. Purge history with `git filter-repo --replace-text <file-listing-the-secrets>` (or BFG), force-push `main`, and ask collaborators to re-clone.
3. Keep secrets only in env vars with a committed `.env.example`; add gitleaks to CI.

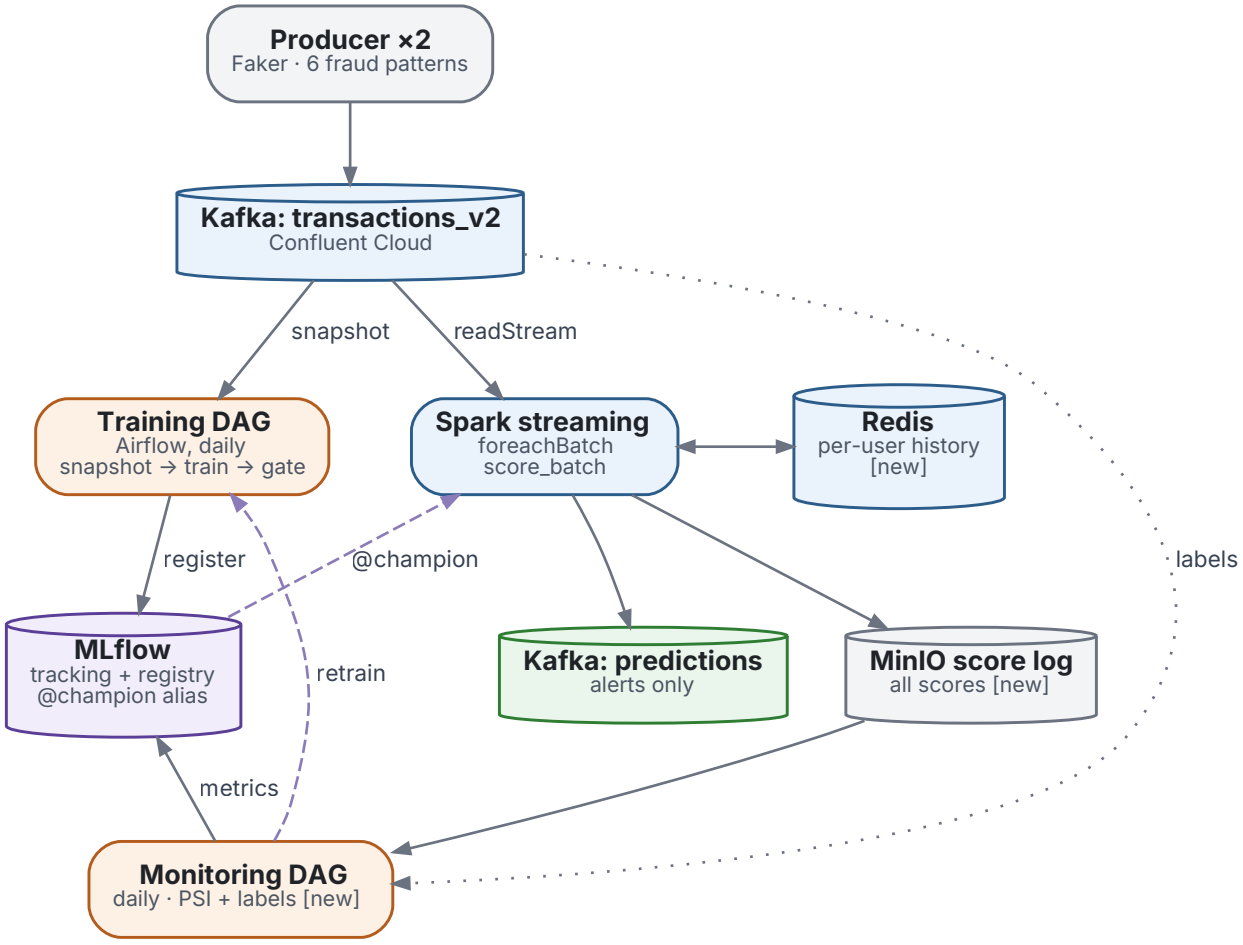
#	Issue	Fix
2	`src/.env`, `src/logs/` (most of the repo's 171 files), `src/models/fraud_detection_model.pkl`, `src/dags/__pycache__/` tracked despite `.gitignore`	`git rm --cached`; the committed pickle is also stale (old features; not the `_v2.pkl` path config uses)
3	README quotes illustrative metrics (AUC-PR 0.95, precision 0.89) as real	Replace with the logged table (§14) and measured results
4	README claims "30+ features," "sub-second latency," "Enterprise Monitoring," "automated deployment," "Daily at 18:00 UTC," "admin/admin," a `fraud_detection_inference` module, and `config/`, `plugins/`, `test.py`	Align each with the code: 30 features, measured latency, monitoring DAG, gate, 00:18 UTC, `airflow/airflow`, real layout
5	DAG: `max_active_runs` in `default_args` (ignored); cron comment says 18:00; typo `validate_envronment`; `provide_context` deprecated in Airflow 2	Fix all four
6	Training: `'f1'` is F2; threshold tuned on the training set; random split; unseeded CV; `random_state` read from a missing config key; `infer_signature(X_train, Y_pred)` mismatched; `GIT_PYTHON_GIT_EXECUTABLE='usr/bin/git'` missing its leading slash	Fixed in A2 (plus the path typo)

#	Issue	Fix
7	Unused config keys: `mlflow.registered_model_name`, `kafka.output_topic`, `spark.shuffle_partition`, duplicate top-level `path`	Wire up or delete
8	Inference: hard-coded 0.70 threshold (training picked ~0.87); `withWatermark` without aggregation; `outputMode("update")` on a stateless query; checkpoint not on a volume; `HEALTHCHECK` on an unserved port; card numbers and IPs in alerts	Fixed in A1, A2, A5
9	Kafka jars baked in **and** downloaded at runtime via `spark.jars.packages` (also pulls unused `mlflow-spark`); build runs `wget --spider google.com`	Keep baked jars only; drop the network check
10	Producer: comment says "target 1.5%" and tests say "1.5–2.5%" (code targets 0.5%); account takeover rarely fires (`compromised_cards` never assigned to users); every delivery logged at INFO; `interval=0.0` floods the topic; two replicas run independent rate controllers	Fix comments; draw compromised cards from `user_cards`; log deliveries at DEBUG; nonzero interval; document per-replica behavior; check `Producer.close()` exists in your confluent-kafka version (`flush()` alone is the documented shutdown)
11	Unpinned requirements; Python 3.12 (Airflow) vs 3.10 (inference); `imblearn` shim	Fixed in A6.3

Diagrams

Every diagram referenced in the text, in order. Each one names the section that describes it and links back to that page.

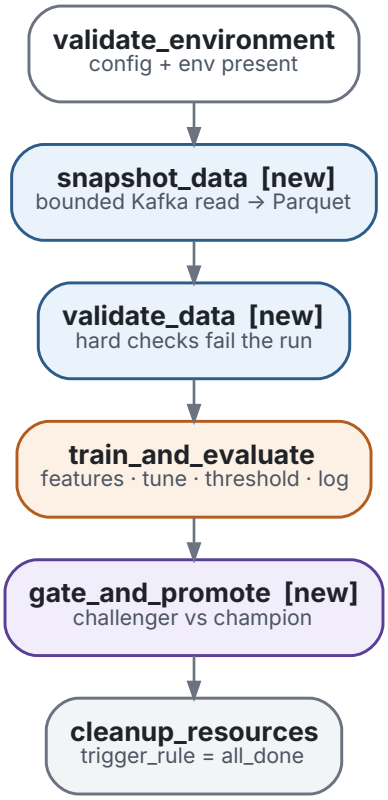
Diagram 1 · System architecture



Training plane (left) and serving plane (right) share only the feature definition and the registered model. [new] = build first.

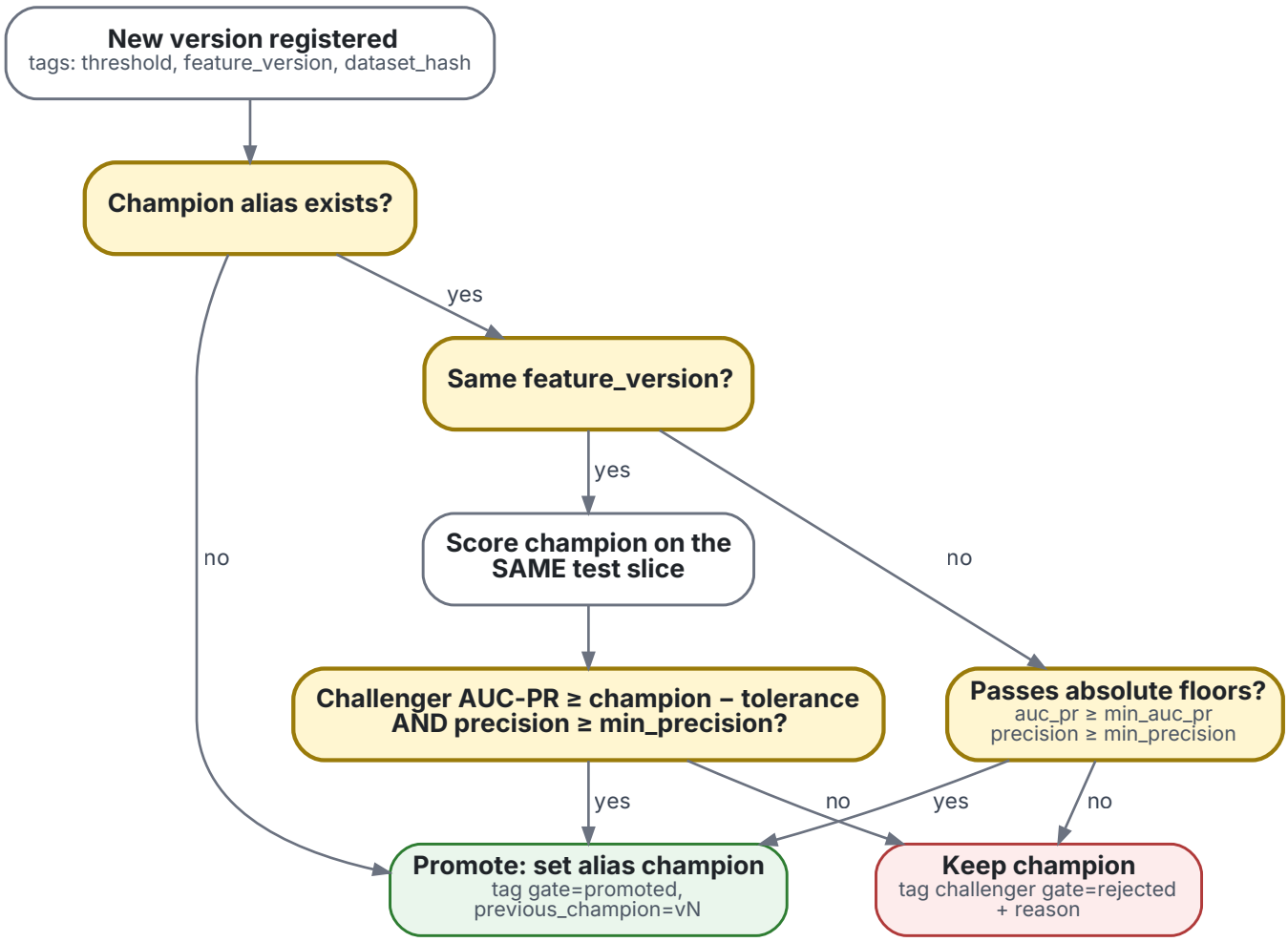
Described in §2 Architecture, page 2

Diagram 2 · Training DAG tasks



Described in §6 The training pipeline (Airflow), page 5

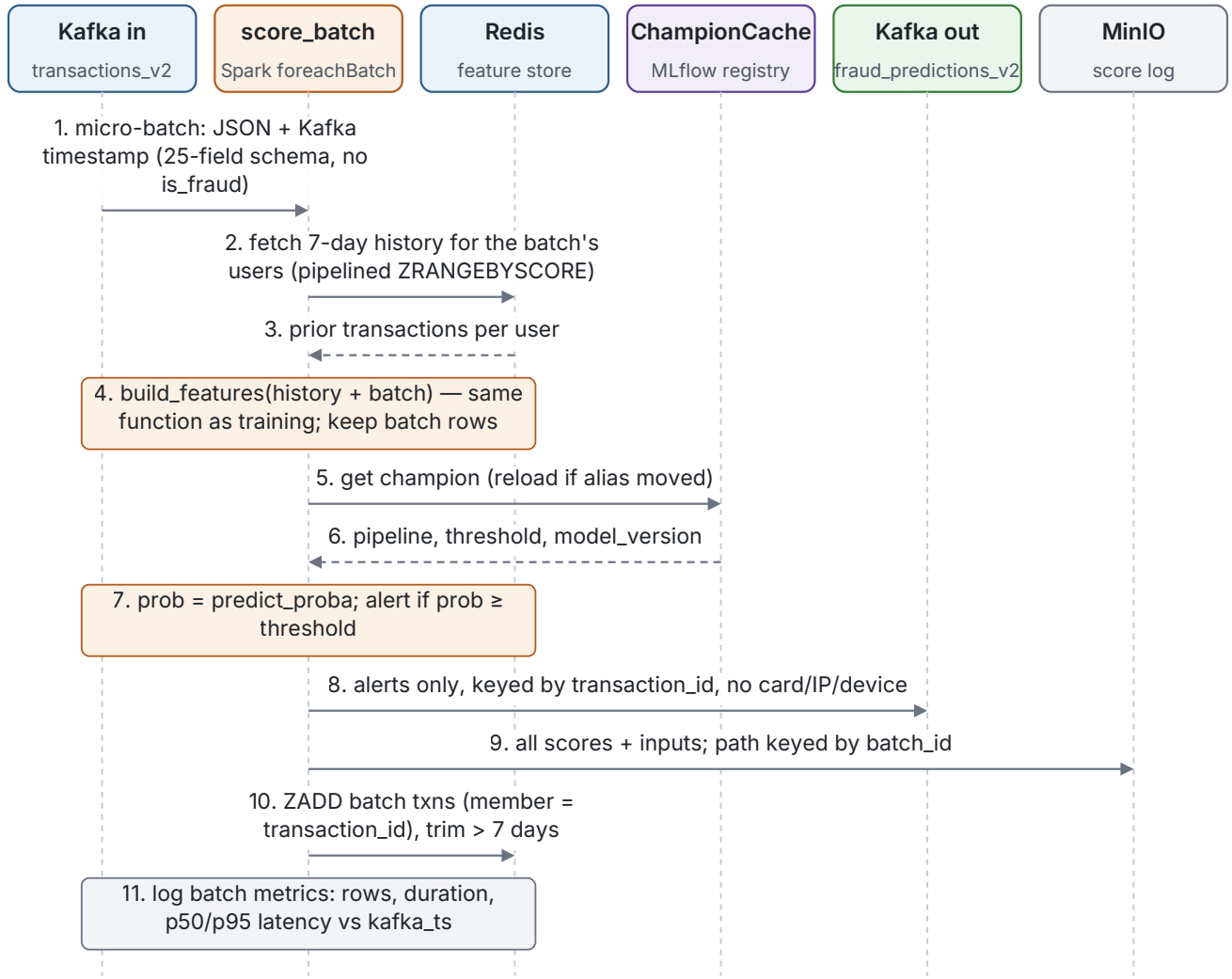
Diagram 3 · Promotion gate: promote_if_better



Runs as its own DAG task in common/registry.py.

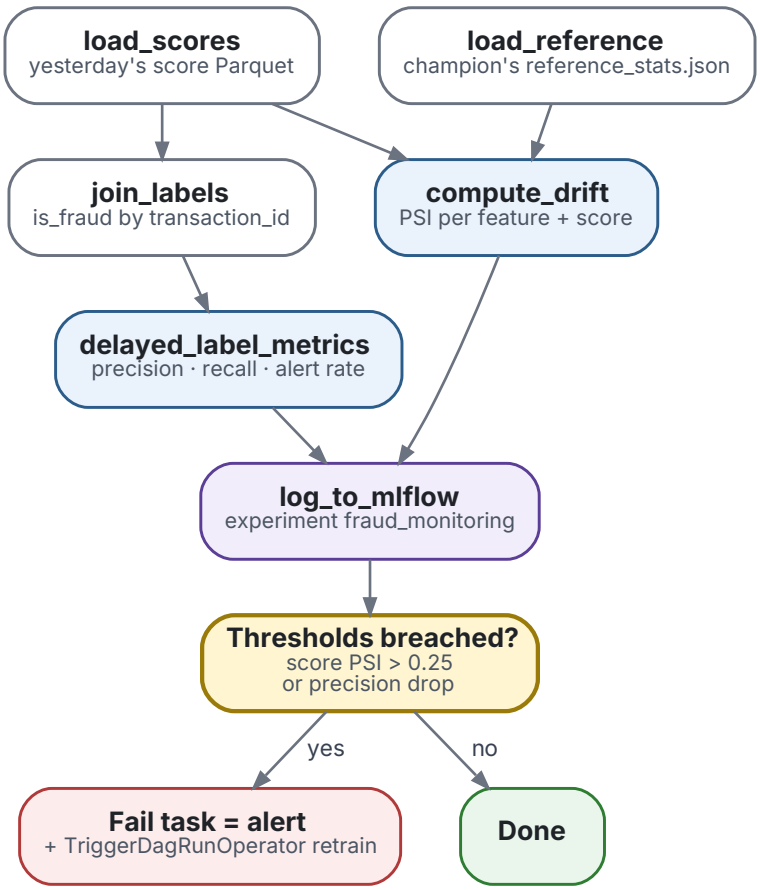
Described in §8 Model registry and the promotion gate, page 6

Diagram 4 · One streaming micro-batch: score_batch



Described in §9 Real-time inference (Spark Structured Streaming), page 7

Diagram 5 · Monitoring DAG



Described in §11 Monitoring and drift, page 8